TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VIỄN THÔNG

BÁO CÁO PBL4

Sinh viên thực hiện

Tên	***Lớp***	***MSSV***
Nguyễn Tấn Minh	22KTMT1	106220224

Giảng viên hướng dẫn
Trần Thị Minh Hạnh

Đà Nẵng, 2025.

MÔ HÌNH CHẨN ĐOÁN ĐỘT QUỴ XÂY DỰNG BẰNG XGBOOST + SHAP

Tóm tắt:

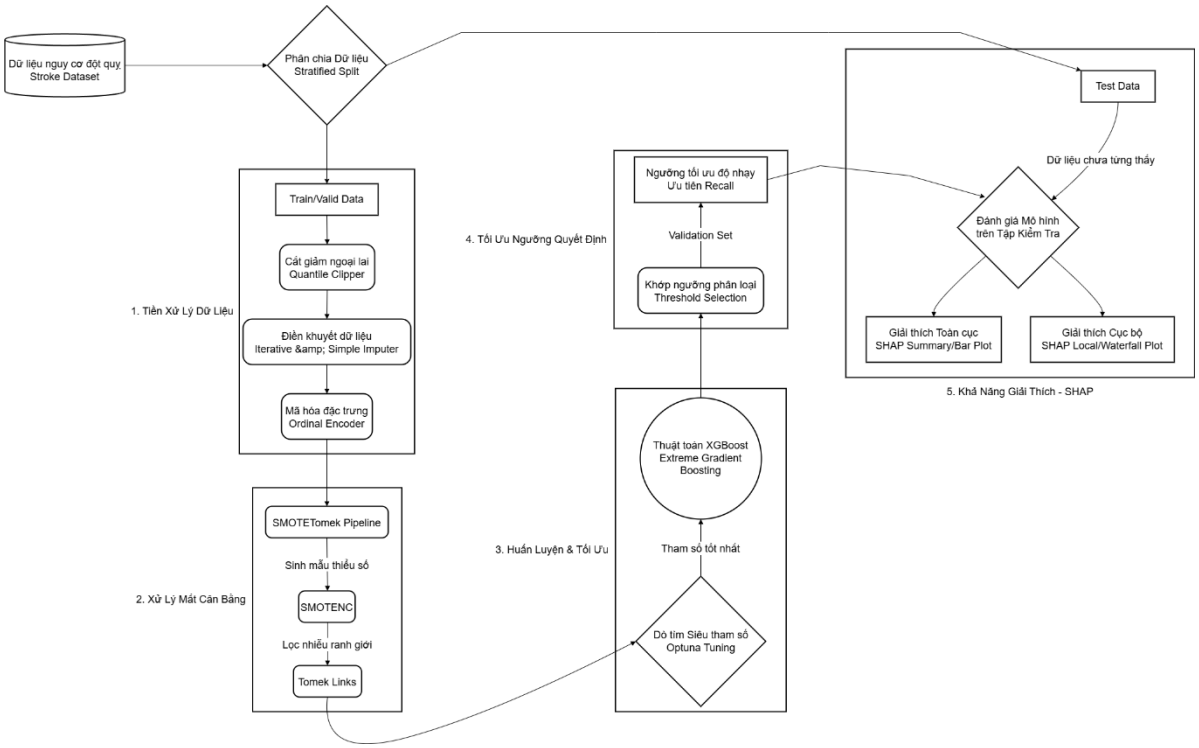
Quy trình: Split dữ liệu → CV loop: preprocess → SMOTETomek (chỉ train fold) → XGBoost → Optuna → refit → test holdout (chỉ 1 lần) → SHAP

Thông tin về các bước xử lí:

1. Xử lí ngoại lệ: Iterative imputer
2. Xử lí mất cân bằng: SMOTETomek (SMOTE + Tomek Links)
3. Tối ưu hóa: Optuna
4. Mô hình hóa: XGBoost
5. Giải thích XAI: SHAP

Mô tả sơ bộ: Mô hình dự đoán nguy cơ đột quỵ được xây dựng trong đồ án này là một đường ống học máy (Machine Learning Pipeline) hoàn chỉnh, được thiết kế theo hướng tiếp cận mô-đun (modular) nhằm đảm bảo tính chính xác, khả năng mở rộng và độ tin cậy khi triển khai trong thực tế. Đồ án không chỉ tập trung vào việc đưa ra dự đoán "Có" hoặc "Không" mắc bệnh, mà còn hướng tới việc giải quyết bài toán "Hộp đen" (Black-box) trong trí tuệ nhân tạo y tế thông qua các kỹ thuật giải thích mô hình.

Flowchart:



Từ khóa:

- [$X_{\cdot j}$]: là toàn bộ các giá trị của feature j ~ {X_{1j}, X_{2j}, …, X_{nj}}.
- Prune/Prunning (cắt tỉa): là cơ chế **loại bỏ các nhánh không cần thiết của cây quyết định**. (Gain<0 với XGBoost).
- Over-sampling: Giảm số lượng mẫu của lớp nhiều mẫu
- Under-sampling: Tăng số lượng mẫu của lớp ít mẫu bằng cách nội suy giữa các điểm gần nhau
- Shrinkage: là cơ chế làm nhỏ bước cập nhật của mỗi cây trong Boosting bằng cách nhân với một hệ số nhỏ η(learning rate).

MỤC LỤC

GIỚI THIỆU 6
TỔNG QUAN DỰ ÁN 7
DỮ LIỆU ĐẦU VÀO 7
1. **Nguồn dữ liệu** 7
2. **Các đặc trưng số - dạng định lượng (Numeric Features)** 8
3. **Các đặc trưng phân loại (Categorical Features)** 8
4. **Phân bố lớp mục tiêu (Target Distribution)** 8
5. **Báo cáo ngoại lai (Outlier Report – phương pháp IQR)** 8
6. **Chia tách dữ liệu (Data Splitting)** 9
TIỀN XỬ LÝ DỮ LIỆU (PREPROCESSING) 9
1. **Xử lý đặc trưng số (Numeric Pipeline)** 9
2. **Xử lý đặc trưng phân loại (Categorical Pipeline)** 10
XỬ LÝ MẤT CÂN BẰNG LỚP (RESAMPLING) 10
1. **SMOTENC (SMOTE for Nominal and Continuous features)** 10
2. **Tomek Links (Under-sampling)** 11
THUẬT TOÁN XGBOOST – NỀN TẢNG TOÁN HỌC 11
1. **Tổng quan** 11
2. **Mô hình Boosting** 11
3. **Chuyển đổi sang xác suất (Sigmoid / Logistic Function)** 12
4. **Hàm mất mát (Loss Function)** 12
5. **Xấp xỉ Taylor Bậc 2 (Second-Order Approximation)** 12
6. **Trọng số tối ưu tại mỗi lá (Optimal Leaf Weight)** 13
7. **Hàm đánh giá phân tách (Split Gain)** 13
8. **Cơ chế Shrinkage (Learning Rate)** 13
9. **Subsampling** 13
10. **Bảng tham số mặc định (Default Parameters)** 13
TỐI ƯU HÓA SIÊU THAM SỐ (OPTUNA) 14
1. **Framework Optuna** 14
2. **Phương pháp lấy mẫu: TPESampler** 14
3. **Cơ chế Pruning: MedianPruner** 14
4. **Không gian tìm kiếm (Search Space)** 14

5. **Cross-Validation trong Optuna** 15
6. **Kết quả tối ưu Optuna** 15
LỰA CHỌN NGƯỠNG QUYẾT ĐỊNH (THRESHOLD) 15
1. **Bài toán** 15
2. **Chiến lược: recall_at_min_precision** 16
3. **Kết quả chọn ngưỡng (trên tập Validation)** 16
CÁC CHỈ SỐ ĐÁNH GIÁ 16
1. **Ma trận nhầm lẫn (Confusion Matrix)** 16
2. **Precision (Độ chính xác dương)** 16
3. **Recall / Sensitivity (Độ nhạy)** 16
4. **Specificity (Độ đặc hiệu)** 17
5. **F1-Score** 17
6. **ROC-AUC** 17
7. **Chỉ số chính: PR-AUC (Average Precision)** 17
8. **Brier Score** 17
GIẢI THÍCH MÔ HÌNH BẰNG SHAP 17
1. **Nền tảng lý thuyết – SHAP Values** 17
2. **TreeExplainer** 18
3. **Tính chất Additivity** 18
4. **Các biểu đồ SHAP được tạo** 18
5. **Ví dụ giải thích cục bộ (Local Explanation)** 20
KẾT QUẢ THỰC NGHIỆM 20
1. **Baseline – Cross-Validation (5 Fold, threshold = 0.5)** 20
2. **Kết quả sau Optuna Tuning (trên tập Test)** 21
3. **Chi tiết Confusion Matrix (Tập Test)** 21
TÀI LIỆU THAM KHẢO 22

GIỚI THIỆU

Đột quỵ (Stroke), hay còn gọi là tai biến mạch máu não, hiện đang là một trong những thách thức y tế nghiêm trọng nhất trên toàn cầu. Theo Tổ chức Y tế Thế giới (WHO), đột quỵ là nguyên nhân gây tử vong đứng thứ hai và là nguyên nhân hàng đầu gây tàn tật lâu dài ở người trưởng thành. Bệnh thường xảy ra đột ngột khi dòng máu cung cấp cho não bị gián đoạn hoặc suy giảm đáng kể, khiến tế bào não chết đi chỉ trong vài phút do thiếu oxy và dưỡng chất.

Sự nguy hiểm của đột quỵ nằm ở chỗ nó thường tiến triển âm thầm và "tấn công" bệnh nhân một cách bất ngờ. Dù vậy, theo nhiều nghiên cứu y khoa, lên đến 80% các ca đột quỵ có thể được phòng ngừa nếu các yếu tố nguy cơ nội tại (huyết áp cao, bệnh tim mạch, tuổi tác, lượng đường huyết) và ngoại tại (thói quen hút thuốc, béo phì, môi trường sống) được phát hiện, theo dõi và can thiệp kịp thời.

sự bùng nổ của Trí tuệ Nhân tạo (AI) - cụ thể là các thuật toán Học máy (Machine Learning) - đã mở ra một kỷ nguyên mới trong "y tế dự phòng kĩ thuật số".

Không giống như con người, các mô hình học máy như XGBoost, Random Forest hay Deep Learning có khả năng "học" và trích xuất những quy luật ẩn (hidden patterns) phi tuyến tính từ hàng nghìn, hàng triệu hồ sơ bệnh án. AI có thể phát hiện sự tương quan tinh vi giữa các chỉ số sinh tồn (như việc một bệnh nhân vừa lớn tuổi, vừa hút thuốc nhưng lượng đường huyết ở mức cận biên rủi ro) để đưa ra chỉ số xác suất đột quỵ với độ chính xác và tốc độ vượt trội.

Tuy nhiên, rào cản lớn nhất ngăn cản AI được tin dùng rộng rãi tại các bệnh viện chính là bài toán "Hộp đen" (Black-box) – mô hình đưa ra kết quả dự đoán đúng nhưng lại không thể giải thích *vì sao* lại có kết quả đó. Bác sĩ không thể chỉ dựa vào một con số phần trăm khô khan để quyết định phác đồ điều trị cho bệnh nhân.

Mục tiêu:

Nhận thức được thực trạng và nhu cầu cấp thiết đó, đồ án này được thực hiện nhằm xây dựng một "Hệ thống dự báo nguy cơ đột quỵ ứng dụng Máy học có tính giải thích".

Bằng cách tận dụng sức mạnh phân loại mạnh mẽ của thuật toán XGBoost, kết hợp giải quyết bài toán dữ liệu y tế mất cân bằng bằng SMOTETomek, hệ thống hướng tới việc tối đa hóa khả năng nhận diện sớm ca bệnh. Điểm đột phá của dự án nằm ở việc tích hợp công nghệ SHAP (SHapley Additive exPlanations), giúp "minh bạch hóa" quá trình ra quyết định của AI. Thay vì chỉ dự đoán tổng quát, mô hình sẽ phân tích chi tiết từng hồ sơ bệnh

nhân, chỉ ra cụ thể yếu tố nào (chỉ số BMI, đường huyết hay mỡ máu,...) đang đóng vòng lớn nhất gây nguy cơ đột quỵ cho chính ca bệnh đó.

Lí do chọn XGBoost + SHAP:

[1] Dựa vào repository trên github của pjaiswalusf: *Stroke Prediction Model*. Tác giả sử dụng tận 5 loại mô hình từ 5 loại thuật toán khác nhau để so sánh: GradientBoosting, AdaBoost, RandomForest, XGBoost, LogisticRegression, tác giả đã dùng Optuna để chọn ra Random Forest là loại tốt nhất, nhưng theo ghi nhận của bài báo "Performance evaluation of hybrid WOA-XGBoost, GWO-XGBoost and BO-XGBoost models to predict blast-induced ground vibration" Y. Qiu, J. Zhou, M. Khandelwal, H. Yang, P. Yang, và C. Li, "Performance evaluation of hybrid WOA-XGBoost, GWO-XGBoost and BO-XGBoost models to predict blast-induced ground vibration" *Engineering with Computers*, vol. 38, số 5, tháng 10 2022, doi: 10.1007/s00366-021-01393-9.

[2] Y. Qiu, J. Zhou, M. Khandelwal, H. Yang, P. Yang, và C. Li, "Performance evaluation of hybrid WOA-XGBoost, GWO-XGBoost and BO-XGBoost models to predict blast-induced ground vibration" *Engineering with Computers*, vol. 38, số 5, tháng 10 2022, doi: 10.1007/s00366-021-01393-9.

[3] Y. Qiu, J. Zhou, M. Khandelwal, H. Yang, P. Yang, và C. Li, "Performance evaluation of hybrid WOA-XGBoost, GWO-XGBoost and BO-XGBoost models to predict blast-induced ground vibration" *Engineering with Computers*, vol. 38, số 5, tháng 10 2022, doi: 10.1007/s00366-021-01393-9.

[4] thì XGBoost là ông vua của dữ liệu dạng bảng (.csv) và điểm khuyết trong Repo này là không cấu hình cụ thể và chi tiết cho XGBoost khiến nó không hoạt động tốt. Thế nên bài làm này là để tối ưu hóa việc sử dụng XGBoost trong việc chuẩn đoán đột quỵ.

TỔNG QUAN DỰ ÁN

Hệ thống XGBoost-SHAP Stroke Prediction Model là một pipeline học máy hoàn chỉnh (end-to-end), được thiết kế để dự đoán nguy cơ đột quỵ não từ dữ liệu lâm sàng của bệnh

nhân. Bài toán được mô hình hóa như bài toán phân loại nhị phân (Binary Classification): dự đoán bệnh nhân có bị đột quỵ (stroke = 1) hay không (stroke = 0).

Quy trình tổng quan (Pipeline Flow):

CSV Data → Data Audit → Train/Valid/Test Split → Preprocessing → SMOTE-Tomek Resampling → XGBoost Training → Optuna Hyperparameter Tuning → Threshold Selection → Final Evaluation → SHAP Explainability → Model Export (.joblib)

DỮ LIỆU ĐẦU VÀO

1. **Nguồn dữ liệu**

Bộ dữ liệu: healthcare-dataset-stroke-data.csv

Thông số	Giá trị
Tổng số mẫu	5.110 bản ghi
Số cột	12 cột (bao gồm id và stroke)
Dữ liệu trùng lặp	0 hàng

2. **Các đặc trưng số - dạng định lượng (Numeric Features)**

Tên cột	Ý nghĩa	Ghi chú
age	Tuổi bệnh nhân	Từ 0.32 đến 82 tuổi
hypertension	Tăng huyết áp	Nhị phân: 0 hoặc 1
heart_disease	Bệnh tim	Nhị phân: 0 hoặc 1
avg_glucose_level	Mức đường huyết trung bình	Liên tục
bmi	Chỉ số khối cơ thể	Có 3.93% giá trị thiếu

3. **Các đặc trưng phân loại (Categorical Features)**

Tên cột	Ý nghĩa	Các giá trị
gender	Giới tính	Male, Female, Other
ever_married	Đã kết hôn	Yes, No
work_type	Loại công việc	Private, Self-employed, Govt_job, children, Never_worked
Residence_type	Loại cư trú	Urban, Rural
smoking_status	Tình trạng hút thuốc	formerly smoked, never smoked, smokes, Unknown

4. **Phân bố lớp mục tiêu (Target Distribution)**

Lớp	Số lượng	Tỷ lệ
Không đột quỵ (0)	4.861	95.13%
Đột quỵ (1)	249	4.87%
Tỷ lệ thiểu số		~1:19.5

Đây là bộ dữ liệu **cực kỳ mất cân bằng** (imbalanced), với lớp thiểu số (stroke) chiếm chưa tới 5%. Điều này ảnh hưởng trực tiếp đến chiến lược mô hình hóa và lựa chọn chỉ số đánh giá.

5. **Báo cáo ngoại lai (Outlier Report – phương pháp IQR)**

Ngoại lai được phát hiện bằng phương pháp Interquartile Range (IQR):

$$IQR = Q_3 - Q_1$$

$$Lower\ Bound = Q_1 - 1.5 \times IQR$$

$$Upper\ Bound = Q_3 + 1.5 \times IQR$$

Đặc trưng	Q1	Q3	IQR	Số ngoại lai	Tỷ lệ
age	25.0	61.0	36.0	0	0%
avg_glucose_level	77.25	114.09	36.85	627	12.27%
bmi	23.5	33.1	9.6	110	2.15%

6. **Chia tách dữ liệu (Data Splitting)**

Dữ liệu được chia làm 3 tập bằng hàm train_test_split từ scikit-learn, sử dụng phân tầng (stratified split) để duy trì tỷ lệ lớp:

Tập	Tỷ lệ	Mục đích
Train	65% (~3.322 mẫu)	Huấn luyện mô hình + Cross-validation
Validation	15% (~766 mẫu)	Chọn ngưỡng quyết định (threshold)
Test	20% (~1.022 mẫu)	Đánh giá cuối cùng

Cách tính tỷ lệ validation điều chỉnh (vì chia 2 bước):

$$val_size_adjusted = \frac{val_size}{(1 - test_size)} = \frac{0.15}{(1 - 0.20)} = 0.1875$$

TIỀN XỬ LÝ DỮ LIỆU (PREPROCESSING)

Hệ thống sử dụng ColumnTransformer của scikit-learn để xử lý riêng biệt đặc trưng số và đặc trưng phân loại.

1. **Xử lý đặc trưng số (Numeric Pipeline)**

   Bước 1: Cắt ngoại lai bằng Quantile (QuantileClipper)

   Đây là transformer tùy chỉnh (custom transformer) nhằm giới hạn giá trị cực đoan. Thay vì xóa ngoại lai, phương pháp này ép chúng về biên, giữ nguyên số lượng mẫu.

   Fit (học ngưỡng từ tập train):

   $$L_j = Percentile(X_{\cdot j}, 0.01) \quad — \quad Percentile\ 1\%\ cho\ mỗi\ cột\ j$$

   $$U_j = Percentile(X_{\cdot j}, 0.99) \quad — \quad Percentile\ 99\%\ cho\ mỗi\ cột\ j$$

   Transform (áp dụng cắt):

   $$X_clipped(i,j) = clip(X_{ij}, L_j, U_j) = max(L_j, min(X_{ij}, U_j))$$ [5]

   Bước 2: Mã hóa thứ bậc (OrdinalEncoder)

   Sử dụng IterativeImputer (sklearn, experimental) – phương pháp MICE (Multiple Imputation by Chained Equations).

   Nguyên lý toán học: Xem mỗi đặc trưng có giá trị thiếu như một biến phụ thuộc, và các đặc trưng khác là biến độc lập. Lặp lại quá trình ước lượng [6]:

   $$x_j^{(t)} = f_j(X_{-j}^{(t-1)} + \varepsilon_j)$$

   Trong đó:

   - X_{-j} là ma trận các đặc trưng còn lại
   - f_j là mô hình hồi quy (mặc định: BayesianRidge)
   - t là chỉ số vòng lặp (tối đa 20 vòng)
   - Giá trị khởi tạo ban đầu: Median (trung vị)

2. **Xử lý đặc trưng phân loại (Categorical Pipeline)**

   Bước 1: Điền giá trị thiếu (SimpleImputer)

   Sử dụng chiến lược most_frequent (mode): thay giá trị thiếu bằng giá trị xuất hiện nhiều nhất. [5] [6]

   $$x_imputed(i,j) = mode(x_{\cdot j})$$

   Bước 2: Mã hóa thứ bậc (OrdinalEncoder)

Mỗi giá trị phân loại được gán một số nguyên duy nhất [7]:

$encode(c_k) = k, \quad k \in \{0, 1, 2, \ldots, K-1\}$

Với giá trị chưa gặp trong tập train → gán giá trị đặc biệt -1 (unknown_value=-1).

Lưu ý: XGBoost là thuật toán dựa trên cây quyết định, nên OrdinalEncoder hoạt động hiệu quả mà không cần One-Hot Encoding. Cây quyết định có thể học ngưỡng phân tách trên các giá trị ordinal này. [7]

XỬ LÝ MẤT CÂN BẰNG LỚP (RESAMPLING)

Pipeline sử dụng SMOTETomek từ thư viện imbalanced-learn, kết hợp hai kỹ thuật:
Bước 1 – SMOTENC: Over-sampling lớp thiểu số
Bước 2 – Tomek Links: Under-sampling - Làm sạch ranh giới quyết định

1. **SMOTENC (SMOTE for Nominal and Continuous features)**

   SMOTENC (SMOTE for Nominal and Continuous features) là biến thể của SMOTE được thiết kế để xử lý đồng thời cả đặc trưng liên tục lẫn đặc trưng phân loại (sau khi đã encode). Thuật toán [8]:

   - Cho mỗi mẫu thiểu số x_i, tìm k láng giềng gần nhất (k-nearest neighbors) cùng lớp
   - Chọn ngẫu nhiên một láng giềng x_nn
   - Tạo mẫu tổng hợp:

   $$x_new = x_i + \lambda.(x_nn - x_i), \text{ với } \lambda \sim U(0, 1). \quad (SMOTE)$$

   Với SMOTENC, đặc trưng phân loại được xử lý đặc biệt [8]:

   - Khoảng cách Euclidean có trọng số trung vị của std cho đặc trưng phân loại
   - Giá trị phân loại của mẫu mới = giá trị majority vote trong k láng giềng

2. **Tomek Links (Under-sampling)**

   Sau khi SMOTE tạo mẫu mới, Tomek Links loại bỏ các cặp mẫu "mâu thuẫn" ở ranh giới lớp.

   Định nghĩa Tomek Link: Cặp (x_i, x_j) là Tomek Link nếu [9]:

   - $x_i \in$ lớp 0, $x_j \in$ lớp 1
   - $d(x_i, x_j) < d(x_i, x_k)$ với mọi $x_k \neq x_j$ thuộc lớp 1
   - $d(x_i, x_j) < d(x_j, x_m)$ với mọi $x_m \neq x_i$ thuộc lớp 0

   Các mẫu trong Tomek Links được loại bỏ → làm sạch ranh giới quyết định.

   Quan trọng: Resampling chỉ được thực hiện trên tập TRAIN thông qua imblearn.pipeline.Pipeline (github). Tập validation và test KHÔNG bị ảnh hưởng bởi resampling – đảm bảo đánh giá trung thực.

THUẬT TOÁN XGBOOST – NỀN TẢNG TOÁN HỌC

1. **Tổng quan**

   XGBoost (eXtreme Gradient Boosting) là thuật toán ensemble learning dựa trên kỹ thuật gradient boosting với cây quyết định (CART) làm base learner. Dự án sử dụng biến thể phân loại nhị phân (binary:logistic). [7]

2. **Mô hình Boosting**

   Mô hình là tổng của T cây quyết định [7]:

$$\hat{y}_i = \sum_{t=1}^{T} f_t(x_i), \qquad với\ f_t \in F$$

   Trong đó:

   - f_t là cây quyết định thứ t
   - F là không gian của tất cả cây CART có thể
   - $\hat{y}_i$ là log-odds (raw prediction) cho mẫu i

3. **Chuyển đổi sang xác suất (Sigmoid / Logistic Function)**

   Vì objective = "binary:logistic", output cuối cùng là xác suất thông qua hàm sigmoid [10]:

$$P(y_i = 1 \mid x_i) = \sigma(\hat{y}_i) = \frac{1}{1 + e^{-\hat{y}_i}}$$

4. **Hàm mất mát (Loss Function)**

   XGBoost tối thiểu hóa hàm mục tiêu tổng quát [10]:

$$L = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t)$$

   Thành phần 1 – Binary Cross-Entropy Loss (logloss) [10]:

$$l(y_i, \hat{y}_i) = -[y_i.\ln\big(\sigma(\hat{y}_i)\big) + (1 - y_1).\ln\big(1 - \sigma(\hat{y}_i)\big)]$$

   Thành phần 2 – Regularization Term (Chính quy hóa) [10]:

$$\Omega(f_t) = \gamma.|T_t| + \frac{\lambda}{2}\sum_{j} w_j^2 + a\sum_{j} |w_j^2|$$

   Trong đó:

   - $|T_t|$ = số lá (leaf nodes) của cây thứ t
   - w_j = trọng số (weight/score) tại lá j
   - γ (gamma) = hình phạt cho mỗi lá mới → kiểm soát độ phức tạp cây
   - λ (reg_lambda) = L2 regularization trên trọng số lá

- α (reg_alpha) = L1 regularization trên trọng số lá

5. **Xấp xỉ Taylor Bậc 2 (Second-Order Approximation)**

   Tại vòng lặp thứ t, XGBoost sử dụng xấp xỉ bậc 2 của hàm loss [7]:

$$L^{(t)} \approx \sum_{i=1}^{n} \left[g_i . f_t(x_i) + \frac{1}{2} h_i . f_t^2(x_i) \right] + \Omega(f_t)$$

   Gradient (đạo hàm bậc 1) [7]:

$$g_i = \frac{\partial l}{\partial \hat{y}_i} = \sigma\left(\hat{y}_i^{(t-1)}\right) - y_i$$

   Hessian (đạo hàm bậc 2) [7]:

$$h_i = \frac{\partial^2 l}{\partial \hat{y}_i^2} = \sigma\left(\hat{y}_i^{(t-1)}\right).(1 - \sigma(\hat{y}_i^{(t-1)})$$

6. **Trọng số tối ưu tại mỗi lá (Optimal Leaf Weight)**

   Cho lá j chứa tập mẫu I_j, trọng số tối ưu [7]:

$$w_j^* = -\sum_{i \in I_j} g_i \Big/ \sum_{i \in I_j} h_i + \lambda$$

7. **Hàm đánh giá phân tách (Split Gain)**

   Khi xem xét phân tách một nút thành hai nhánh trái (I_L) và phải (I_R) [7]:

$$Gain = \frac{1}{2}\left[\left(\sum_{i \in I_L} g_i\right)^2 \Big/ \sum_{i \in I_L} (h_i + \lambda) + \left(\sum_{i \in I_R} g_i\right)^2 \Big/ \sum_{i \in I_R} (h_i + \lambda) - \left(\sum_{i \in I} g_i\right)^2 \Big/ \sum_{i \in I} (h_i + \lambda) \right] - \gamma$$

   Nếu Gain ≤ 0 → KHÔNG phân tách (gamma hoạt động như ngưỡng tối thiểu cho gain).

8. **Cơ chế Shrinkage (Learning Rate)**

   Sau khi xây dựng cây thứ t, cập nhật dự đoán [7]:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta . f_t(x_i)$$

   Trong đó η (learning_rate) là hệ số co rút:

   - Giảm overfit bằng cách "bước nhỏ" hơn mỗi lần cập nhật
   - Cho phép các cây sau sửa lỗi tốt hơn

9. **Subsampling**

Tham số	Ý nghĩa	Phạm vi
subsample	Tỷ lệ mẫu ngẫu nhiên cho mỗi cây	[0.6, 1.0]
colsample_bytree	Tỷ lệ đặc trưng ngẫu nhiên cho mỗi cây	[0.6, 1.0]

10. **Bảng tham số mặc định (Default Parameters)**

Tham số	Giá trị mặc định	Ý nghĩa
n_estimators	400	Số cây
max_depth	4	Độ sâu tối đa mỗi cây
learning_rate	0.05	Hệ số shrinkage
min_child_weight	2.0	Tổng Hessian tối thiểu trong một lá
subsample	0.9	Tỷ lệ mẫu cho mỗi cây
colsample_bytree	0.9	Tỷ lệ đặc trưng cho mỗi cây
gamma	0.0	Min gain để phân tách
reg_alpha (L1)	0.0	Regularization L1
reg_lambda (L2)	1.0	Regularization L2
objective	binary:logistic	Hàm mục tiêu
eval_metric	logloss	Metric đánh giá

TỐI ƯU HÓA SIÊU THAM SỐ (OPTUNA)

1. **Framework Optuna**

   Hệ thống sử dụng Optuna – framework tối ưu hóa Bayesian hiện đại – để tìm bộ siêu tham số tốt nhất. [11]

2. **Phương pháp lấy mẫu: TPESampler**

   Tree-structured Parzen Estimator (TPE) hoạt động theo nguyên lý [12]:

   B1: Chia các bộ tham số đã thử thành hai nhóm dựa trên performance [12]:

   - *l(x)*: phân phối của các tham số cho kết quả TỐT (trên ngưỡng)
   - *g(x)*: phân phối của các tham số cho kết quả XẤU (dưới ngưỡng)

   B2: TPE tối đa hóa tỷ lệ [12]:

   $$EI(x) \propto \frac{l(x)}{g(x)} \quad \textit{(TPE)}$$

   hoặc viết đầy đủ hơn theo cách trình bày của TPE:

   $$EI(x) = C\,\frac{l(x)}{g(x)} \quad \textit{(TPE)}$$

   Ưu tiên vùng tham số mà các trial tốt tập trung, tránh vùng cho kết quả xấu.

3. **Cơ chế Pruning: MedianPruner**

   MedianPruner dừng sớm (prune) các trial kém hiệu quả [11]:

   - Tại mỗi bước trung gian, so sánh kết quả hiện tại với trung vị (median) của tất cả trial trước đó

- Nếu kết quả tệ hơn trung vị → prune (dừng trial đó)
- n_warmup_steps = 2: không prune trong 2 bước đầu

4. Không gian tìm kiếm (Search Space)

Tham số	Phân phối	Phạm vi
n_estimators	Uniform Integer	[150, 900]
max_depth	Uniform Integer	[3, 8]
learning_rate	Log-Uniform	[0.001, 0.2]
min_child_weight	Uniform Float	[1.0, 12.0]
subsample	Uniform Float	[0.6, 1.0]
colsample_bytree	Uniform Float	[0.6, 1.0]
gamma	Uniform Float	[0.0, 8.0]
reg_alpha	Log-Uniform	[10^{-8}, 5.0]
reg_lambda	Log-Uniform	[0.001, 20.0]

5. Cross-Validation trong Optuna

Mỗi trial đánh giá bằng Stratified K-Fold CV với K = 5, [11]:

$$Score_trial = \frac{1}{K}\sum_{k=1}^{K} PR - AUC(y_val_k, \hat{p}_val_k)$$

6. Kết quả tối ưu Optuna

Kết quả:

Thông số	Giá trị
Tổng số trial	30
Số trial bị pruned	11 (37%)
Best trial	Trial #9
Best CV PR-AUC	0.1728

Bộ siêu tham số tối ưu:

Tham số	Giá trị tối ưu
n_estimators	618
max_depth	4
learning_rate	0.00140

min_child_weight	4.421
subsample	0.730
colsample_bytree	0.892
gamma	5.100
reg_alpha	0.522
reg_lambda	0.107

LỰA CHỌN NGƯỠNG QUYẾT ĐỊNH (THRESHOLD)

1. **Bài toán**

   Mặc định, phân loại nhị phân sử dụng ngưỡng t = 0.5:

   $\hat{y} = 1 \text{ nếu } P(y = 1 \mid x) \geq t$

   $\hat{y} = 0 \text{ nếu } P(y = 1 \mid x) < t$

   Tuy nhiên với dữ liệu mất cân bằng, ngưỡng 0.5 thường không tối ưu => Sẽ điều chỉnh cho bản cập nhật tới khoảng 0.1 - 0.3.

2. **Chiến lược: recall_at_min_precision**

   Dự án sử dụng chiến lược "Tối đa hóa Recall với ràng buộc Precision tối thiểu" [10]:

   $$t^* = \arg[\max_t Recall(t)] \ \boldsymbol{subject\ to}\ Precision(t)$$

   Cách hoạt động:

   (1) Duyệt 99 ngưỡng từ 0.01 đến 0.99 (trên tập validation)

   (2) Tính Precision, Recall, F1 tại mỗi ngưỡng

   (3) Lọc các ngưỡng có Precision ≥ 15%

   (4) Trong các ngưỡng khả thi, chọn ngưỡng cho Recall cao nhất

   (5) Nếu hòa Recall, ưu tiên F1 cao hơn, rồi threshold thấp hơn

3. **Kết quả chọn ngưỡng (trên tập Validation)**

Thông số	Giá trị
Ngưỡng được chọn	0.57
Precision tại ngưỡng	17.05%
Recall tại ngưỡng	40.54%
F1 tại ngưỡng	24.00%

CÁC CHỈ SỐ ĐÁNH GIÁ

1. **Ma trận nhầm lẫn (Confusion Matrix)** [10]

	Dự đoán = 0	**Dự đoán = 1**
Thực tế = 0	TN (True Negative)	FP (False Positive)
Thực tế = 1	FN (False Negative)	TP (True Positive)

2. **Precision (Độ chính xác dương)**

$$Precision = TP / (TP + FP)$$ [10]

Trả lời cho câu hỏi: Trong số các mẫu được dự đoán là đột quỵ, bao nhiêu phần trăm thực sự bị đột quỵ?

3. **Recall / Sensitivity (Độ nhạy)**

$$Recall = TP / (TP + FN)$$ [10]

Trả lời cho câu hỏi: Trong số các bệnh nhân thực sự bị đột quỵ, mô hình phát hiện được bao nhiêu phần trăm?

4. **Specificity (Độ đặc hiệu)**

$$Specificity = TN / (TN + FP)$$ [10]

Trả lời cho câu hỏi: Trong số bệnh nhân không bị đột quỵ, bao nhiêu phần trăm được mô hình xác nhận đúng?

5. **F1-Score**

Trung bình điều hòa của Precision và Recall:

$$F_1 = 2 \cdot (Precision \times Recall) / (Precision + Recall)$$ [10]

6. **ROC-AUC**

ROC Curve vẽ đồ thị True Positive Rate (TPR) vs False Positive Rate (FPR) [13]:

$$ROC - AUC = \int_0^1 TPR(FPR)dFPR$$

Với:

$TPR = Recall = TP / (TP + FN)$

$FPR = FP / (FP + TN) = 1 - Specificity$

ROC-AUC = 1.0: phân loại hoàn hảo. ROC-AUC = 0.5: đoán ngẫu nhiên.

7. **Chỉ số chính: PR-AUC (Average Precision)** [14]

$$AP = \sum_{k=1}^{n} (R_k - R_{k-1}) P_k$$

Tại sao chọn PR-AUC thay vì ROC-AUC?

=> Với dữ liệu mất cân bằng nặng (5% positive), ROC-AUC có thể cho kết quả lạc quan sai vì TN rất lớn làm FPR luôn nhỏ. PR-AUC tập trung vào lớp thiểu số và phản ánh chính xác hơn khả năng phát hiện đột quỵ.

8. **Brier Score** [15]

$$Brier = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2$$

Brier Score = 0: xác suất dự đoán hoàn hảo. Brier Score nhỏ hơn → xác suất hiệu chuẩn tốt hơn.

GIẢI THÍCH MÔ HÌNH BẰNG SHAP

1. **Nền tảng lý thuyết – SHAP Values**

SHAP (SHapley Additive exPlanations) dựa trên lý thuyết Shapley value từ game theory. Giá trị SHAP của đặc trưng j cho mẫu x được định nghĩa [16]:

$$\phi_j(x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!}[f(S \cup \{j\}) - f(S)]$$

Trong đó:

F = tập tất cả đặc trưng

S = tập con đặc trưng không chứa j

f(S) = giá trị kỳ vọng của mô hình khi chỉ dùng đặc trưng trong S

*** S ⊆ F \ {j}: mọi tập con không chứa feature j

***|S| hoặc |F|: là số phần tử ở trong tập đó

Ý nghĩa trực quan: SHAP value = đóng góp trung bình biên của đặc trưng j vào dự đoán, xét trên mọi tổ hợp đặc trưng có thể

2. **TreeExplainer**

Dự án sử dụng shap.TreeExplainer – phiên bản tối ưu cho mô hình dựa trên cây, với [16]:

- Interventional perturbation: đánh giá importance bằng cách can thiệp (thay đổi) đặc trưng
- Background dataset: tối đa 300 mẫu (lấy mẫu từ tập train)
- Explain dataset: tối đa 500 mẫu (từ tập test)

3. **Tính chất Additivity**

Đặc tính cộng gộp (Additivity) của hệ thống SHAP [16]:

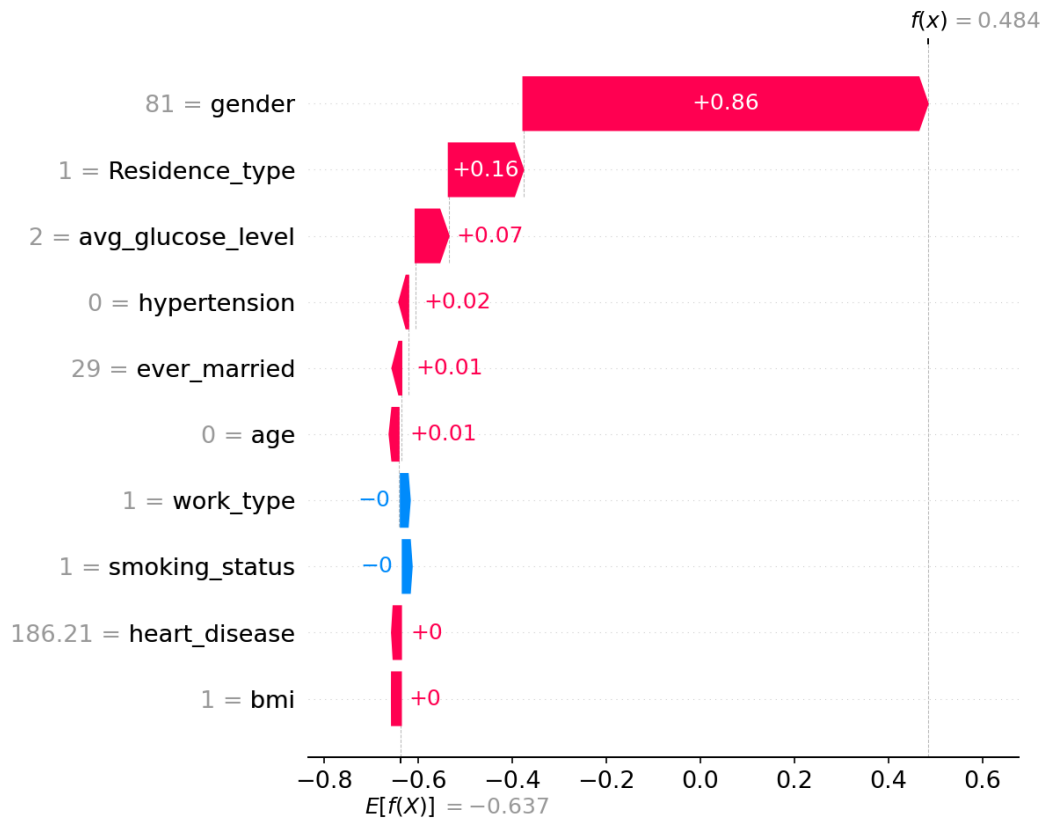
$$f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j(x)$$

Trong đó:

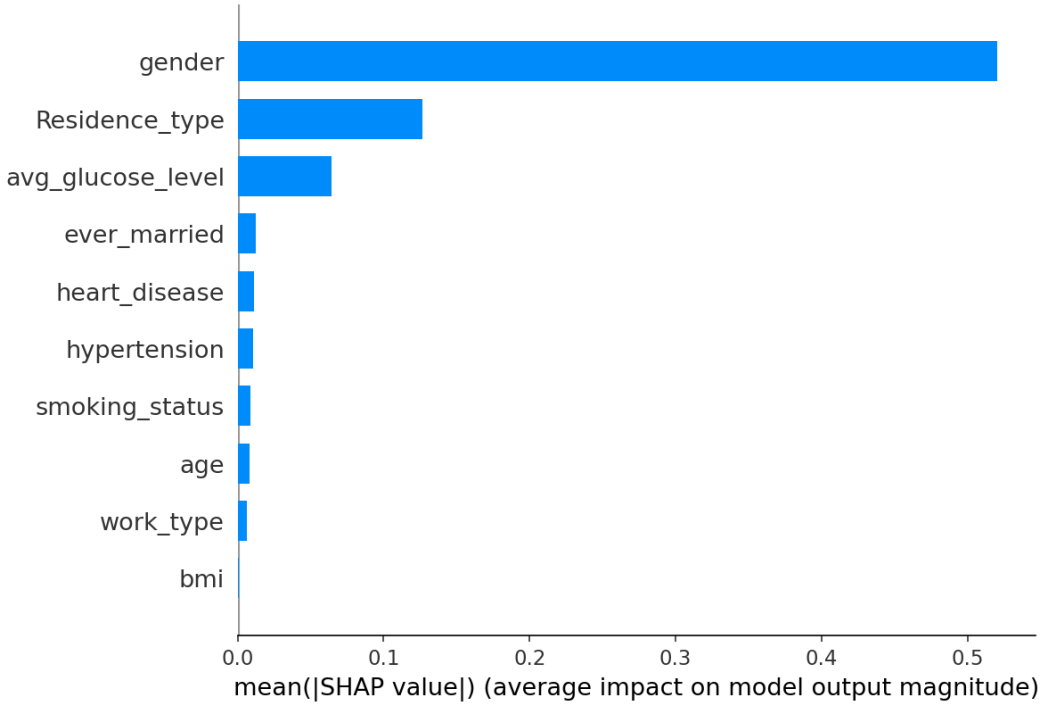
- f(x) = output mô hình (log-odds)
- ϕ_0 = base value (giá trị kỳ vọng trung bình) = −0.6365
- $\Sigma\phi_j$ = tổng đóng góp của các đặc trưng

4. Các biểu đồ SHAP được tạo

- Beeswarm Plot (shap_summary_beeswarm.png): Phân bố SHAP values cho tất cả mẫu



- Bar Plot (shap_summary_bar.png): Trung bình tuyệt đối SHAP value (feature importance)



- Waterfall Plot (shap_local_waterfall.png): Giải thích cụ thể cho mẫu có nguy cơ cao nhất

5. Ví dụ giải thích cục bộ (Local Explanation)

Mẫu có xác suất đột quỵ dự đoán cao nhất (61.86%):

Đặc trưng	Giá trị SHAP	Hướng tác động
age (81 tuổi)	+0.860	↑ Tăng nguy cơ mạnh nhất
Residence_type	+0.159	↑ Tăng nhẹ
avg_glucose_level	+0.071	↑ Tăng nhẹ
hypertension	+0.015	↑ Tăng rất nhẹ
work_type	−0.005	↓ Giảm nhẹ
smoking_status	−0.004	↓ Giảm nhẹ

KẾT QUẢ THỰC NGHIỆM

1. **Baseline – Cross-Validation (5 Fold, threshold = 0.5)**

Chỉ số	Mean ± Std
PR-AUC	0.1277 ± 0.0243
ROC-AUC	0.7643 ± 0.0410
Recall	0.2104 ± 0.0832
Precision	0.1299 ± 0.0545
F1	0.1603 ± 0.0656
Specificity	0.9269 ± 0.0106
Brier Score	0.0778 ± 0.0063

2. **Kết quả sau Optuna Tuning (trên tập Test)**

Chỉ số	Giá trị	Thay đổi so với Baseline
PR-AUC	0.2148	+68.2%
ROC-AUC	0.8025	+5.0%
Recall	0.3800	+80.6%
Precision	0.2209	+70.1%
F1	0.2794	+74.3%
Specificity	0.9311	Tương đương
Brier Score	0.1437	Tăng (xấu hơn)
Threshold	0.5700	recall_at_min_precision

3. **Chi tiết Confusion Matrix (Tập Test)**

	Dự đoán = 0	Dự đoán = 1	Tổng

Thực tế = 0	TN = 905	FP = 67	972
Thực tế = 1	FN = 31	TP = 19	50
Tổng	936	86	1.022

Phân tích:

- Trong 50 bệnh nhân đột quỵ thực tế → phát hiện đúng 19/50 (Recall = 38%)
- Trong 86 cảnh báo đột quỵ → có 19 trường hợp đúng (Precision = 22.1%)
- 31 bệnh nhân đột quỵ bị bỏ sót (False Negative) – đây là rủi ro chính trong y tế

Vấn đề cần cải thiện:

Vấn đề	Chi tiết	Độ nghiêm trọng
PR-AUC thấp (0.215)	Mô hình chỉ nắm bắt được một phần nhỏ pattern đột quỵ	Cao
Recall thấp (38%)	Bỏ sót 62% bệnh nhân đột quỵ – rủi ro y tế nghiêm trọng	Rất cao
Brier Score tệ hơn	0.1437 vs baseline 0.078 → xác suất chưa hiệu chuẩn tốt	Trung bình
Bộ dữ liệu nhỏ	5.110 mẫu, chỉ 249 positive → không đủ để học deep patterns	Cao
Thiếu Feature Engineering	Chưa tạo đặc trưng tương tác (age × glucose, BMI groups)	Trung bình

TÀI LIỆU THAM KHẢO

[4] Y. Qiu, J. Zhou, M. Khandelwal, H. Yang, P. Yang, và C. Li, "Performance evaluation of hybrid WOA-XGBoost, GWO-XGBoost and BO-XGBoost models to predict blast-induced ground vibration" *Engineering with Computers*, vol. 38, số 5, tháng 10 2022, doi: 10.1007/s00366-021-01393-9.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python" *Journal of Machine Learning Research*, vol. 12, 2011.

[6] S. van Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate Imputation by Chained Equations in R" *Journal of Statistical Software*, vol. 45, no. 3, 2011.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System" in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique" *Journal of Artificial Intelligence Research*, vol. 16, 2002.

[9] I. Tomek, "Two Modifications of CNN" *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-6, no. 11, 1976.

[10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework" in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019.

[12] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization" in *Advances in Neural Information Processing Systems*, vol. 24, 2011.

[13] T. Fawcett, "An introduction to ROC analysis" *Pattern Recognition Letters*, vol. 27, no. 8, 2006.

[14] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets" *PLoS One*, vol. 10, no. 3, p. e0118432, 2015.

[15] G. W. Brier, "Verification of forecasts expressed in terms of probability" *Monthly Weather Review*, vol. 78, no. 1, 1950.

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions" in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, 2017.